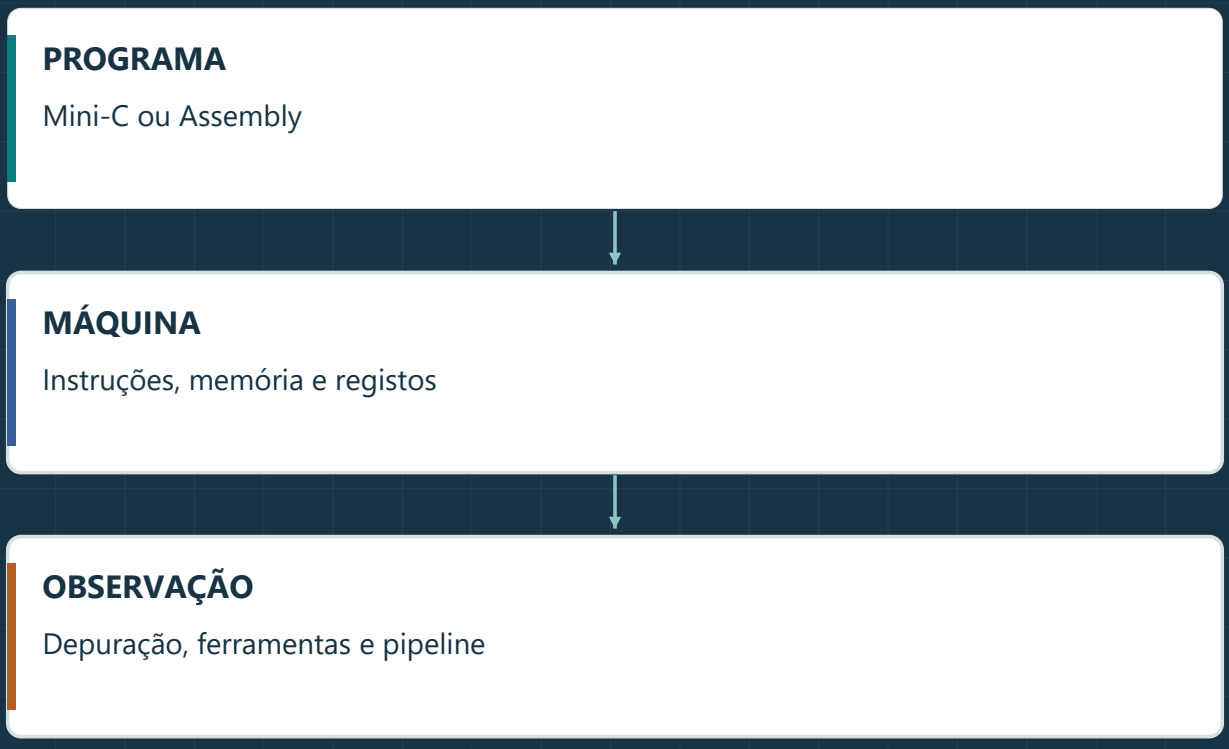
webMARS

Atlas do projeto

Propósito, filosofia e arquitetura

DA IDEIA À MÁQUINA OBSERVÁVEL



Um ambiente educativo que aproxima o código do comportamento da máquina e torna as diferenças visíveis.

LEITURA DO PRODUTO 0.6.6 / EDIÇÃO 1.0

29 de outubro de 2026 | PT-PT | Fonte: 098494f17a36

Uma plataforma, quatro escalas

O webMARS liga programação, execução, exploração de arquitetura e partilha de trabalho. Cada escala acrescenta contexto ao mesmo programa.

04 Partilhar e ensinar

Projetos Cloud, publicações, exemplos e manual.

03 Explorar arquitetura

20 ferramentas: pipeline, cache, dispositivos e representações.

02 Executar e compreender

Montar, executar, depurar e recuar sobre o estado MIPS.

01 Construir o programa

Assembly MIPS e Mini-C, com projetos e vários ficheiros.

QUEM GANHA COM ESTA LIGAÇÃO

Aprendente

Vê como o código altera memória e registos.

Docente

Partilha exemplos e compara hipóteses em aula.

Instituição

Aloja a sua instalação e escolhe os serviços Online.

Fronteira do produto

Mini-C é um subconjunto de C. O runtime é um simulador educativo; o sistema de ficheiros é virtual e os serviços Online são opcionais.

Fontes: README.md; tools/tools.json; manual/content/shared.mjs; webmars-config.js.

Princípios que orientam decisões

Esta leitura da filosofia resulta das fronteiras e dos comportamentos já presentes no projeto. Cada princípio tem uma consequência concreta.

01

Aprender pela observação

Código, instruções, memória, registos e E/S descrevem a mesma máquina. Uma explicação deve poder ser confrontada com o estado visível.

02

Autonomia local

Editar, compilar, montar e executar não exigem conta. A persistência no browser mantém-se sujeita a quotas, falhas e políticas de limpeza.

03

Separar factos de modelos

Uma instrução executada é um facto do runtime. Um ciclo de pipeline pertence a uma configuração e a um fluxo declarados.

04

Preservar o trabalho

Uma falha de gravação é um erro. Eliminar localmente não apaga a cópia Cloud; um conflito exige uma decisão explícita.

05

Compatibilidade demonstrável

A referência MARS, os testes e as diferenças declaradas sustentam as afirmações de compatibilidade. Uma percentagem isolada não prova correção.

06

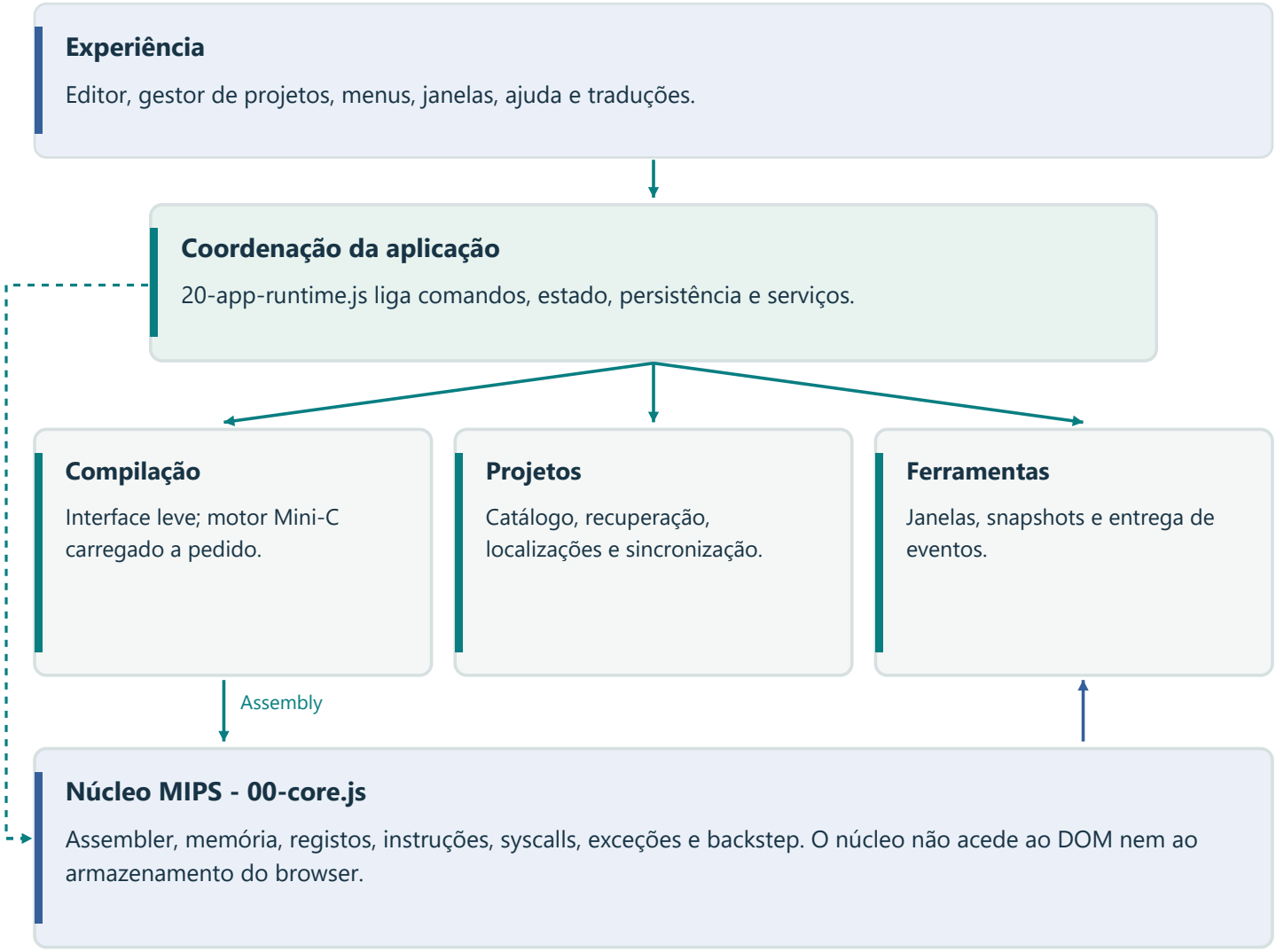
Coerência na experiência

Projetos e localizações partilham a mesma estrutura visual. Temas, teclado e 20 idiomas atravessam os vários fluxos.

Fontes: docs/code-architecture.md; docs/runtime-invariants.md; docs/pipeline-lab.md; TODO_0.7.0.txt.

Como o sistema se organiza

Mapa conceptual dos módulos atuais. As setas mostram ligações de coordenação ou circulação de resultados; a posição não representa a ordem de carregamento.



FRONTEIRA DO BROWSER

Serviços Online opcionais

BackMars: contas, projetos privados, publicações e versões remotas.

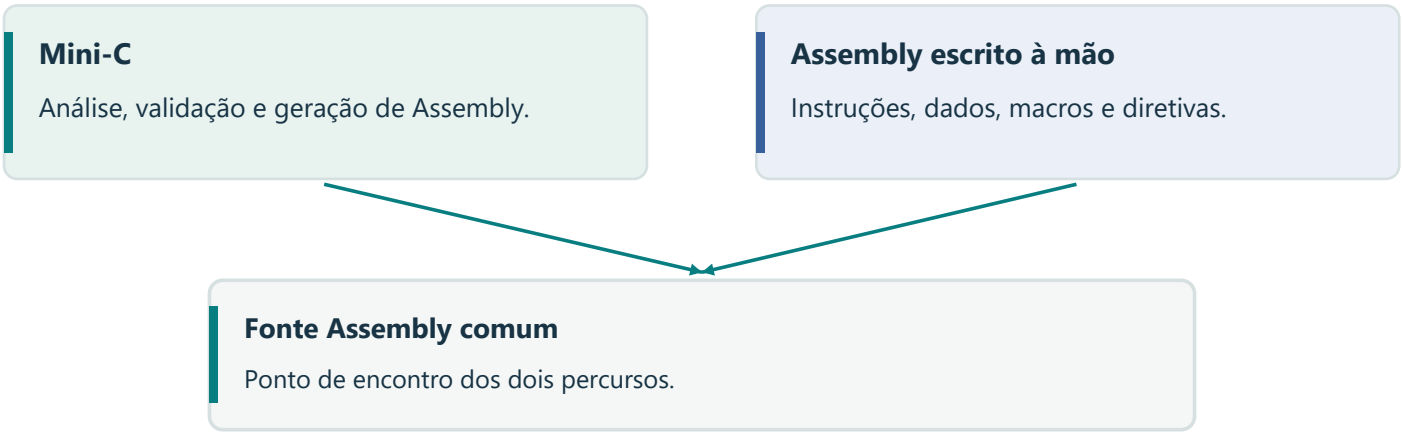
Recolha separada

Métricas de utilização e agregados. Não executa o programa MIPS.

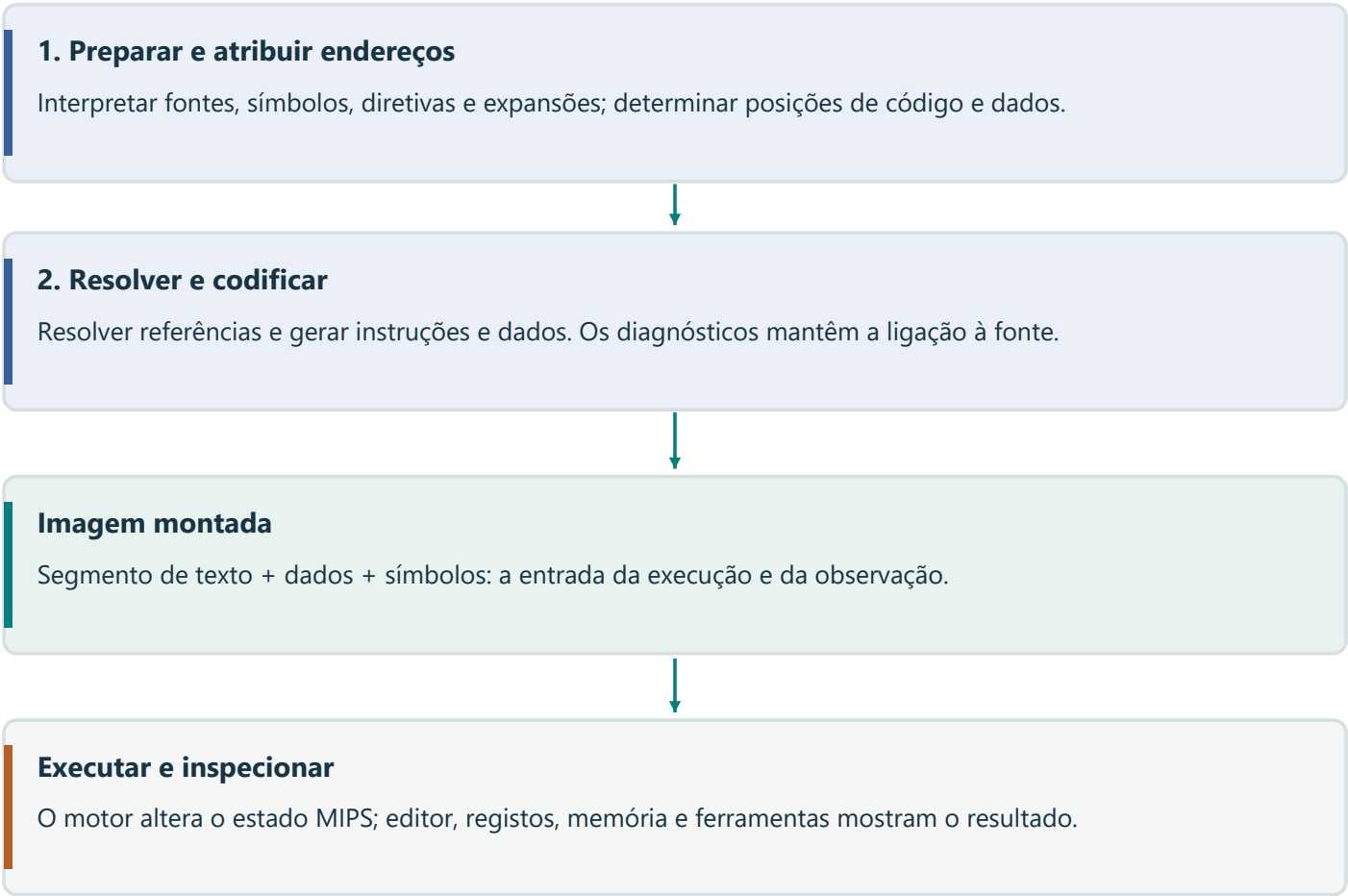
Fontes: assets/js/app.bundle.js; docs/code-architecture.md. O código usa scripts ordenados e globais partilhados.

Da intenção ao estado da máquina

Compilar, montar e executar são operações distintas. O mesmo caminho de montagem recebe tanto Assembly escrito à mão como Assembly gerado por Mini-C.



ALGORITMO DE MONTAGEM / VISÃO SIMPLIFICADA



Um erro de compilação ou de montagem deve ser resolvido na respetiva fase. Montar um programa não equivale a executá-lo.

Fontes: 17-mini-c-interface.js; 17-mini-c-compiler.js; 00-core.js; tests/c-compiler-regression.test.mjs.

Um passo que pode ser explicado

O caminho normal de uma instrução produz alterações observáveis e a informação necessária para recuar. Este esquema omite os desvios de exceção e de espera por input.

Ler e descodificar

Usar PC e imagem montada.

Validar e executar

Aplicar semântica, alinhamento e mapa de memória.

Registar alterações

Capturar deltas inversos dos elementos modificados.

Concluir e notificar

Atualizar estado, histórico, snapshots e eventos.

Histórico inverso

PC anterior

Registos escritos

Bytes de memória

Estado adicional capturado

Limites de passos e de bytes

Passo atrás

Aplicar os deltas inversos pela ordem inversa. As ferramentas que mantêm estado próprio acompanham a reversão através do respetivo contrato.

INVARIANTES QUE PROTEGEM A EXPLICAÇÃO

$zero

Permanece zero.

Memória

Acessos passam pela API.

Snapshot

Consumidores não o alteram.

Fontes: docs/runtime-invariants.md; 00-core.js; tests/runtime-memory-invariants.test.mjs; docs/tool-api.md.

Duas perguntas, dois relógios

O runtime responde «o que fez o programa?». O Pipeline Lab responde «como o modelo distribui essas instruções por ciclos?». Os dois contadores não são equivalentes.

Runtime funcional

Executa a semântica MIPS.
Um passo corresponde a uma instrução.
Produz estado e registo de execução.

Modelo de pipeline

Simula ciclos de uma microarquitetura.
Observa instruções e configurações.
Não altera o estado funcional.

DOIS FLUXOS POSSÍVEIS PARA O MODELO

Estático

Instruções montadas e hipótese declarada para os saltos. Sem valores de execução.

Execução registada

Sequência realmente executada, com leituras, escritas e resultados dos saltos observados.

CINCO ESTÁGIOS / MODELO IN-ORDER

IF Busca → **ID** Descodifica → **EX** Opera → **MEM** Memória → **WB** Conclui

Forwarding, stalls, predição e latência alteram a ocupação dos estágios. O CPI é uma medida do modelo, sob a configuração escolhida.

O que a interpretação deve preservar

Não converter ciclos modelados em velocidade real do browser. Não atribuir resultados reais de saltos ao fluxo estático. O replay utiliza apenas o que foi observado.

Fontes: docs/pipeline-lab.md; tools/pipeline-lab.js; tests/pipeline-execution-trace.test.mjs.

Um projeto, várias localizações

Uma localização descreve onde se consulta ou conserva uma cópia. Uma publicação é um snapshot partilhado; não é a pasta editável do autor.

Projeto = identidade + ficheiros + metadados

Ficheiros e pastas organizam o conteúdo. O projeto é a unidade de trabalho, sincronização e publicação.

Neste dispositivo

Cópia editável e catálogo confirmado no browser.

Nuvem

Documento privado numa conta BackMars, com versão remota.

Biblioteca pública

Publicações e revisões imutáveis; abrir cria uma cópia local.

Sincronização explícita entre cópias

AÇÕES PERTENCEM AO CONTEXTO

Local	Editar, organizar, sincronizar, exportar, reciclar.
Nuvem	Consultar, abrir/sincronizar e gerir a cópia remota.
Publicações	Consultar; o autor gere revisões e arquivo.
Reciclagem	Restaurar ou eliminar definitivamente, conforme a seleção.

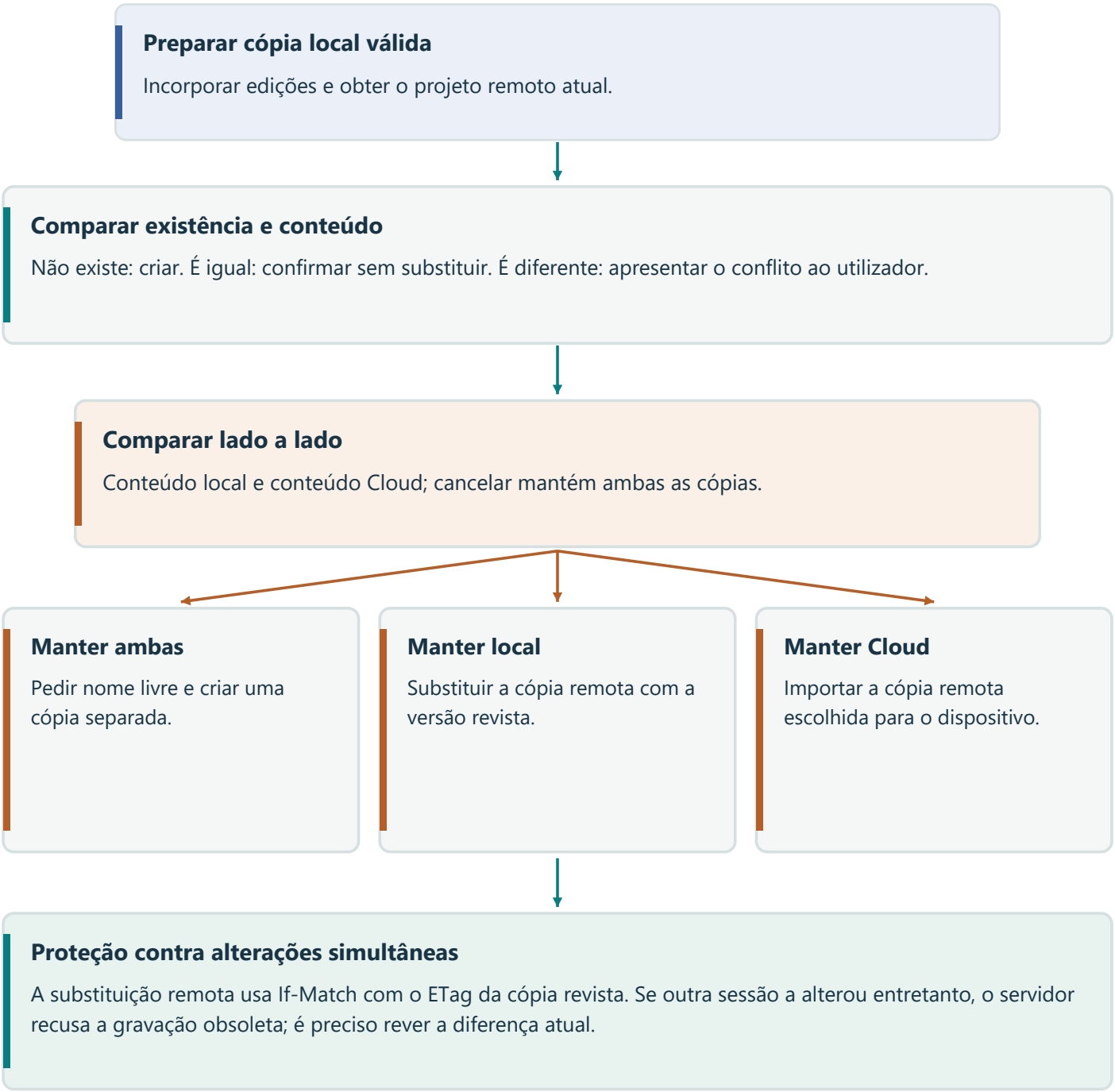
Fonte de verdade atual

O catálogo confirmado em localStorage prevalece, mesmo vazio. IndexedDB funciona como espelho de recuperação; a migração para armazenamento principal ainda não está concluída.

Fontes: 11-ui-file-manager.js; 20-project-repository.js; 20-project-bootstrap.js; 21-project-manager.js.

Sincronizar sem perder decisões

Fluxo conservador da gravação de um projeto na Cloud. O ponto decisivo é comparar a cópia remota atual antes de qualquer substituição.



A sincronização da coleção também trata correspondências e diferenças entre projetos. Este mapa não representa uma fusão automática de linhas: a decisão de conflito escolhe cópias completas.

Fonte: 20-app-runtime.js: saveProjectToCloud e resolveCloudProjectConflict; tests/cloud-sync-contract-regression.test.mjs.

Fronteiras de confiança e de dados

Execução, contas e estatísticas cumprem funções diferentes. A arquitetura deve tornar claro que informação atravessa cada fronteira.



O QUE PODE SAIR DO BROWSER

Projetos: o documento do projeto atravessa a rede quando é sincronizado ou publicado. A publicação torna pública a revisão escolhida.

Utilização: a instalação oficial tem recolha automática configurada. Os eventos não incluem o texto do programa; relatórios públicos mostram agregados.

Local por omissão não significa ausência de rede

O alojamento, a consulta de sessão e os coletores configurados podem originar pedidos. Sessões medidas não são pessoas únicas; a classificação de tráfego é heurística, não prova de humanidade.

Fontes: webmars-config.js; docs/usage-metrics.md; docs/code-architecture.md; manual/content/chapters-operations.mjs.

Uma afirmação precisa de um rasto

A confiança resulta de ligar comportamento, verificação e artefacto publicado. Os números abaixo pertencem aos relatórios desta versão, não a uma promessa sobre todo o MIPS.

8
casos idênticos ao MARS

3
diferenças declaradas

0
diferenças sem explicação

Comparação diferencial de 11 programas com uma referência MARS 4.5 fixada por SHA-256. O conjunto é limitado e as diferenças têm reproduções documentadas.

DA ALTERAÇÃO À PUBLICAÇÃO

1. Verificação funcional

Regressões do motor, compilador, ferramentas, persistência e fluxos.

↓

2. Qualidade da verificação

Cobertura com limites por módulo e mutações selecionadas; 12 defeitos introduzidos foram detetados.

↓

3. Coerência do produto

Idiomas, manual, referências, ligações, carregamento e pacote estático.

↓

4. Artefacto identificável

Versão + build AAAAMMDDhhmm + pacote com checksums; uma base concreta para reproduzir problemas.

Cobertura mede execução de código; não demonstra correção. Testes de mutação avaliam apenas os defeitos introduzidos. Interface visual e condições reais exigem observação própria.

Fontes: docs/mars-conformance.md; docs/mutation-report.md; docs/coverage-report.md; .github/workflows/validate.yml.

O que preservar. O que evoluir.

A evolução deve simplificar a manutenção sem diluir os contratos que tornam a aplicação compreensível e o trabalho recuperável.

Preservar

Núcleo sem dependência do DOM.

Execução funcional separada do timing.

Trabalho local sem conta obrigatória.

Decisões explícitas nos conflitos.

Exemplos e referências verificáveis.

Evoluir - trabalho em aberto

Extrair responsabilidades dos módulos extensos.

Completar, se adotada, a migração para IndexedDB principal.

Manter coerência de UI e contratos durante cada extração.

CRITÉRIOS PARA AVALIAR UMA MUDANÇA

01 Ajuda a compreender o programa ou reduz fricção real?

02 Mantém clara a origem dos dados e das medições?

03 Preserva trabalho perante falha, quota e concorrência?

04 Tem uma fronteira e uma verificação observáveis?

MAPA DE LEITURA DAS FONTES

Arquitetura e contratos	docs/code-architecture.md; docs/tool-api.md
Semântica e modelos	docs/runtime-invariants.md; docs/pipeline-lab.md
Dados e prova	docs/usage-metrics.md; docs/evidence.md
Limites e continuidade	TODO_0.7.0.txt; docs/comparison-matrix.md

As camadas deste atlas são mapas conceptuais dos ficheiros atuais. As propostas de evolução não representam funcionalidades já entregues nem datas de compromisso.